

Autograd:

A hands-on, terminal-driven deep dive into how PyTorch computes gradients — from manual derivatives to the computational graph to real training loops.

01

What Is Autograd?

Autograd (**A**utomatic **G**radient) is PyTorch's engine that records every operation on tensors into a **computational graph**, then uses the **chain rule** to compute derivatives automatically — exactly what gradient descent needs.

1

`requires_grad=True`

Tells PyTorch: "Track all operations on this tensor so I can compute gradients later."

2

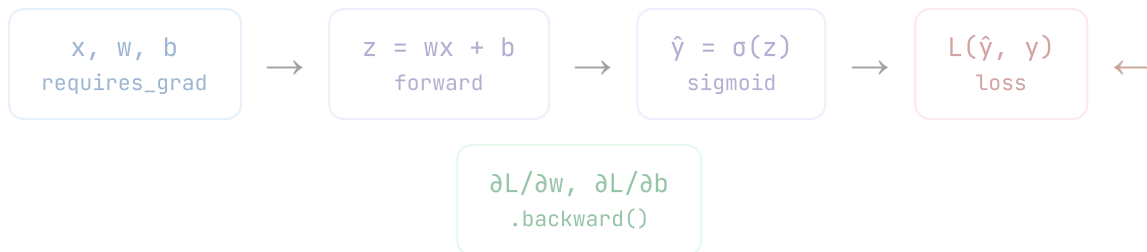
Forward Pass

Perform normal math. PyTorch silently builds a directed acyclic graph (DAG) of every operation.

3

`.backward()`

Walks the graph *backward*, applying the chain rule. Stores results in each tensor's `.grad`.



02 Manual Gradients vs Autograd

Let's first compute gradients *by hand* using the chain rule, then verify with PyTorch's autograd. This shows what's happening under the hood.

manual_gradients.py - PyTorch

```
tensor(6.7012)

# ---- Manual chain rule derivatives ----
# 1. d(loss) / d(y_pred)
>>> d1_dy_pred = (y_pred - y) / (y_pred * (1 - y_pred))

# 2. d(y_pred) / d(z) → sigmoid derivative
>>> dy_pred_dz = y_pred * (1 - y_pred)

# 3. d(z)/d(w) = x, d(z)/d(b) = 1
>>> dz_dw = x
>>> dz_db = 1

# Full chain rule: multiply all the way
>>> d1_dw = d1_dy_pred * dy_pred_dz * dz_dw
>>> d1_db = d1_dy_pred * dy_pred_dz * dz_db

>>> print(f"gradient of loss w.r.t weight (dw): {d1_dw}")
gradient of loss w.r.t weight (dw): tensor(6.6918)
>>> print(f"gradient of loss w.r.t bias (db): {d1_db}")
gradient of loss w.r.t bias (db): tensor(0.9988)

# That was tedious. Let autograd do it for us →
```

► Run Script

03 Autograd – Same Problem, Zero Manual Math

Now the same computation — but PyTorch does all the calculus for us.

autograd_demo.py – PyTorch

```
>>> x = torch.tensor(6.7)
>>> y = torch.tensor(0.8)

# Forward pass – graph is built automatically
>>> z = w * x + b
>>> y_pred = torch.sigmoid(z)
tensor(0.9988, grad_fn=<SigmoidBackward0>)
    ↑ PyTorch remembers this came from sigmoid

# Binary cross-entropy loss
>>> loss = -( y * torch.log(y_pred) + (1 - y) * torch.log(1 - y_pred) )
tensor(6.7012, grad_fn=<NegBackward0>)

# The magic line – one call computes ALL gradients
>>> loss.backward() ← this does all the calculus

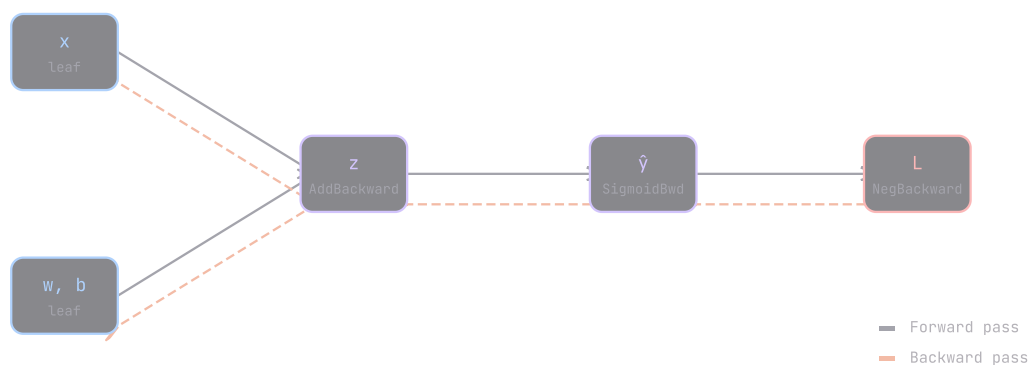
# Read the gradients
>>> print(w.grad)
tensor(6.6918) ← ∂loss/∂w (same as manual!)
>>> print(b.grad)
tensor(0.9988) ← ∂loss/∂b (same as manual!)

# Exact same result, zero manual calculus. That is autograd.
```

► Run Script

04 Inside the Computational Graph

Every tensor operation creates a node. Each node knows its `grad_fn` — the function that created it. When you call `.backward()`, PyTorch traverses this graph in reverse topological order.



graph_inspect.py — PyTorch

```
<SigmoidBackward0> ← created by sigmoid()

# Leaf tensors have no grad_fn, but have .grad after backward
>>> print(x.grad_fn)
None ← x is a leaf tensor

# Trace back through the graph
>>> print(z.grad_fn.next_functions)
((<MulBackward0 ...>, 0), (<AccumulateGrad ...>, 0))
# ↑ w*x           ↑ b (leaf, waiting for .grad)

# After backward, leaf tensors get their gradients
>>> loss = -torch.log(y)
>>> loss.backward()

>>> print(f"w.grad = {w.grad}")
w.grad = tensor(0.2384)
>>> print(f"b.grad = {b.grad}")
b.grad = tensor(0.1192)
>>> print(f"x.grad = {x.grad}")
```

```
x.grad = tensor(0.3577)
```

```
# grad_fn tells you WHERE in the graph a tensor sits
```

► Run Script

05 Gradients for Multiple Values Simultaneously

Autograd works element-wise on tensors. Each element gets its own gradient computed independently.

multi_grad.py – PyTorch

```
# Gradients computed element-wise for tensors
```

```
>>> import torch
```

```
>>> x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
```

```
#  $y = \text{mean}(x^2) = (1 + 4 + 9) / 3 = 4.6667$ 
```

```
>>> y = (x ** 2).mean()
```

```
tensor(4.6667, grad_fn=<MeanBackward0>)
```

```
>>> y.backward()
```

```
#  $dy/d(x_i) = 2 \cdot x_i / n$  (mean divides by  $n=3$ )
```

```
>>> print(x.grad)
```

```
tensor([0.6667, 1.3333, 2.0000])
```

```
#  $\uparrow 2(1)/3 \quad \uparrow 2(2)/3 \quad \uparrow 2(3)/3$ 
```

```
# Each element gets its own gradient!
```

► Run Script



Gradient Accumulation (The Gotcha)

Gradients **accumulate** by default — they don't overwrite. Each `.backward()` call *adds* to `.grad`. This is why you must zero them between training steps.

gradient_accumulation.py — PyTorch

```
>>> y.backward()
>>> print(x.grad)
tensor(4.) ← dy/dx = 2x = 2(2) = 4

# Second backward — without zeroing!
# ⚠ Gradients ACCUMULATE, they don't overwrite
>>> y = x ** 2
>>> y.backward()
>>> print(x.grad)
tensor(8.) ← 4 + 4 = 8 (accumulated!)

# Third time...
>>> y = x ** 2
>>> y.backward()
>>> print(x.grad)
tensor(12.) ← 8 + 4 = 12 (keeps growing!)

# The fix: zero gradients before each backward
>>> x.grad.zero_()
>>> print(x.grad)
tensor(0.) ← clean slate

# Always call .grad.zero_() in training loops!
```

▶ Run Script



Disabling Gradient Tracking

Three ways to stop autograd from tracking operations: `requires_grad_(False)`, `.detach()`, and `torch.no_grad()`.

```
requires_grad_(False)      .detach()      torch.no_grad()
```

disable_requires.py – PyTorch

```
# Method 1: requires_grad_(False)
# Permanently disables tracking on the tensor

>>> import torch

>>> x = torch.tensor(2.0, requires_grad=True)
tensor(2., requires_grad=True)

# Disable it
>>> x.requires_grad_(False)
tensor(2.)
# ↑ requires_grad is gone

# Now backward won't work
>>> y = x ** 2
>>> y.backward()
RuntimeError: element 0 of tensors does not require grad
and does not have a grad_fn

# Use this when you permanently want to stop tracking.
```

► Run Script

08 Putting It All Together – A Real Training Step

This is how autograd works in practice: forward pass → loss → backward → update → zero grad → repeat.

training_loop.py – PyTorch

```
# 4. UPDATE: adjust weights (no grad tracking!)

>>> with torch.no_grad():
>>>     w -= lr * w.grad
>>>     b -= lr * b.grad
```

```

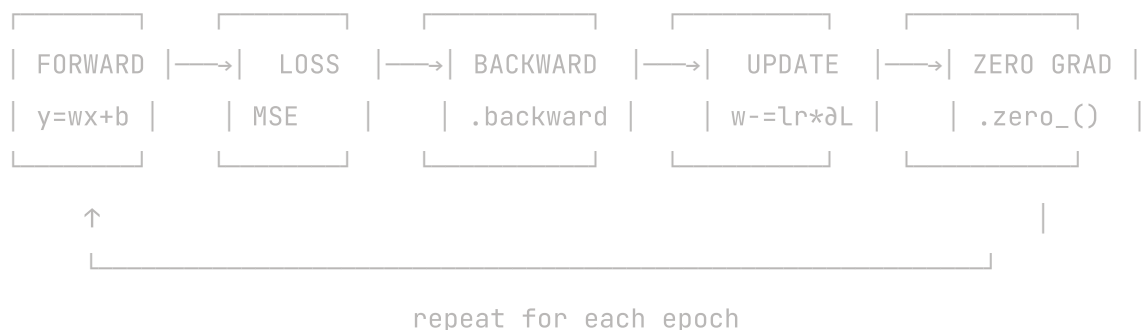
# 5. ZERO: reset gradients for next epoch
>>> w.grad.zero_()
>>> b.grad.zero_()

>>> print(f"Learned: w = {w.item():.4f}, b = {b.item():.4f}")
Learned: w = 2.0000, b = 0.0000

```

It learned $y = 2x$ perfectly! $w \approx 2$, $b \approx 0$

The training step cycle:



► Run Script

09 Quick Reference

CONCEPT	CODE	PURPOSE
Track tensor	<code>x = torch.tensor(1.0, requires_grad=True)</code>	Enable autograd for this tensor
Forward	<code>y = x**2 + 3*x</code>	Builds computation graph automatically
Backward	<code>y.backward()</code>	Compute $\partial y / \partial x$ via chain rule
Read gradient	<code>x.grad</code>	Access the computed gradient value
Zero gradients	<code>x.grad.zero_()</code>	Reset accumulated gradients

CONCEPT	CODE	PURPOSE
Disable tracking	<code>x.requires_grad_(False)</code>	Permanently disables tracking
Detach	<code>z = x.detach()</code>	Clone without grad history
No-grad context	<code>with torch.no_grad():</code>	Disable tracking in a block (inference/updates)

→ **Key takeaway:** Autograd records the forward pass into a graph, then walks it backward using the chain rule to compute how much each input contributed to the output. That's exactly what gradient descent needs to train neural networks.